Radio fantasma · Reparto del equipo

Oct 28, 2026 · @Sandro

Resumen

La sala 2 es una radio con un dial (Slider) que el visitante gira hasta la frecuencia que le da la página HTML de 1.º; al sintonizarla suena una voz y aparece la letra **I**. El proyecto se divide en 6 partes que tocan **archivos distintos**, así que no debería haber conflictos de merge.

Antes de repartir los prompts, sustituid en todo este documento:

- `USUARIO` → el usuario de GitHub del dueño del repo, en minúsculas (paquete `es.medac.USUARIO.tunel`, repo `tunel-terror-javafx-USUARIO`).
- `FRECUENCIA` → la frecuencia acordada con 1.º (por defecto **66.6**).

#	Parte	Quién	Rama	Archivos que le pertenecen
1	Coordinación, estructura base, controlador e integración, README	Tú	`feature/estructura-base`, `feature/integracion`, `docs/readme`	`pom.xml`, `module-info.java`, `App.java`, `SalaController.java`, `README.md`, `.gitignore`
2	Vista FXML con Scene Builder	Compañero 2	`feature/vista-fxml`	`sala.fxml`, `img/radio*.png` (opcional)
3	Estilo CSS oscuro y fuente	Compañero 3	`feature/estilo-css`	`estilos.css`, `estilo/Fuentes.java`, `fuentes/`
4	Lógica del dial y reloj de piedad	Compañero 4	`feature/logica-dial`	`logica/Sintonizador.java`, `logica/RelojPiedad.java`
5	Sonido	Compañero 5	`feature/sonido`	`sonido/GestorSonido.java`, `audio/`
6	Animaciones y susto controlado	Compañero 6	`feature/efectos`	`efectos/Efectos.java`, `img/susto.png`

Calendario (todo antes del jueves 29 a las 08:00):

1. **Miércoles hasta las 14:00** — Parte 1: repo, commit inicial, rama `develop` y estructura base con todos los archivos "esqueleto" ya compilando. Nadie empieza hasta que esto esté en `develop`.

2. **14:00–20:00** — Partes 2 a 6 en paralelo, cada una en su rama, con su Pull Request a `develop`.
3. **20:00–23:00** — Revisión y merge de los PR, integración final en `SalaController`, prueba completa con `mvn clean javafx:run`.
4. **23:00–00:30** — README con capturas y créditos, commit final y merge a `main`.
5. **Jueves 07:45** — Simulacro. Después solo commits `fix:` hasta el viernes a las 08:00.

Flujo de Git

`main` solo recibe la entrega; todo el trabajo entra en `develop` mediante Pull Requests que revisa y fusiona el coordinador (parte 1). Nunca `push --force`, `rebase` de ramas ya subidas ni "Squash" / "Rebase and merge": la profesora contrasta el historial.

Coordinador (una sola vez):

1. Crear el repo **público** `tunel-terror-javafx-USUARIO` y añadir a los 5 compañeros como colaboradores.
2. Primer commit en `main` con el mensaje exacto `chore: inicio sala del terror` (solo `.gitignore` y un README mínimo). Push.
3. `git switch -c develop` y push de `develop`. En GitHub, poner `develop` como rama por defecto para que los PR apunten ahí solos.
4. Rama `feature/estructura-base` con el esqueleto completo (todos los archivos del contrato, compilando y arrancando). PR a `develop` y merge. **Avisar al grupo**: ya podéis empezar.

Cada compañero:

1. `git clone https://github.com/USUARIO/tunel-terror-javafx-USUARIO.git` y `git switch develop` → `git pull`.
2. `git switch -c feature/<su-rama>`.
3. Tocar **solo sus archivos**. Comprobar que arranca con `mvn clean javafx:run` (para salir de la pantalla completa: **Ctrl+Shift+Q**).
4. Commits pequeños y convencionales: `feat:`, `style:`, `fix:`, `refactor:`, `docs:`, `chore:`. Ejemplo: `feat: dial invertido y cálculo de señal`. Mínimo 2 commits por persona.
5. `git push -u origin feature/<su-rama>` y abrir un Pull Request **hacia** **`develop`**. En la descripción: qué hace, cómo probarlo y los **créditos** de recursos usados (sonidos, fuentes, imágenes con su licencia y URL).
6. Si el coordinador pide cambios, nuevos commits en la misma rama y push (el PR se actualiza solo).

Al fusionar un PR (coordinador): usar "Create a merge commit" y **editar el título** para que sea convencional, por ejemplo `feat: integra sonido del dial (#5)`. Así todo el historial cumple el formato.

Palabra del día: cuando la profesora la dicte, el coordinador la añade como comentario en `SalaController.java` en ese mismo momento, en un commit `chore: palabra del dia`, y push.

Entrega final (coordinador):

1. En `develop`, el último commit con el mensaje exacto `docs: entrega final tunel del terror` (README terminado). Push.
2. Llevarlo a `main` sin crear commit de merge, para que ese sea de verdad el último: `git switch main` → `git merge --ff-only develop` → `git push`.
3. Entregar en Canvas la URL del repo.

Requisitos de la rúbrica que esto cubre: ⩾6 commits convencionales, ⩾4 pushes en horas distintas, `SalaController.java` cambiado en ⩾3 commits (esqueleto, integración y palabra del día), commits inicial y final exactos.

Contrato del proyecto

Cada archivo tiene un solo dueño; el número entre paréntesis es la parte. Las firmas públicas de abajo **no se cambian**: el esqueleto de la parte 1 ya las trae vacías, y cada uno rellena las suyas por dentro. Se puede añadir todo lo privado que haga falta.

```
tunel-terror-javafx-USUARIO/
├── pom.xml  README.md  .gitignore                (1)
└── src/main/
    ├── java/
    │   ├── module-info.java                      (1)
    │   └── es/medac/USUARIO/tunel/
    │       ├── App.java                          (1)
    │       ├── SalaController.java               (1)
    │       ├── estilo/Fuentes.java               (3)
    │       ├── logica/Sintonizador.java          (4)
    │       ├── logica/RelojPiedad.java           (4)
    │       ├── sonido/GestorSonido.java          (5)
    │       └── efectos/Efectos.java              (6)
    └── resources/es/medac/USUARIO/tunel/
        ├── sala.fxml                             (2)
        ├── estilos.css                           (3)
        ├── fuentes/                              (3)
        ├── audio/                                (5)
        └── img/   radio*.png (2) · susto.png (6)
```

Nodos de `sala.fxml`

La raíz es un `StackPane` para que la letra y el susto se superpongan a la radio. Los nodos marcados como ocultos empiezan con `visible="false"`.

fx:id	Tipo	Para qué	Clase CSS
`raiz`	StackPane (raíz)	Fondo; tiembla al acertar	`raiz`
`panelRadio`	VBox	Carcasa de la radio con todo lo demás dentro	`radio`
`lblTitulo`	Label	Nombre de la emisora fantasma	`titulo`
`lblInstruccion`	Label	«Gira el dial hasta la frecuencia…» (se entiende en 5 s)	`instruccion`
`lblFrecuencia`	Label	La frecuencia que marca el dial, p. ej. `87.3 MHz`	`frecuencia`
`sldDial`	Slider	El dial	`dial`
`pbSenal`	ProgressBar	Intensidad de la señal de 0 a 1	`senal`
`lblAyuda`	Label	Pista de ayuda a los 60 s (oculto)	`ayuda`

fx:id	Tipo	Para qué	Clase CSS
`lblLetra`	Label	La letra **I** en grande (oculto, encima de todo)	`letra`
`imgSusto`	ImageView	Cara fantasma del susto (oculto, encima de todo)	—

El controlador añade la clase `sintonizada` a `raiz` cuando aciertan y la quita al reiniciar.

APIs

```
// (3) estilo/Fuentes.java
public final class Fuentes {
    public static void cargar();   // Font.loadFont de cada .ttf de fuentes/;
si falla, avisa por consola
}

// (4) logica/Sintonizador.java  — Java puro, sin JavaFX
public class Sintonizador {
    public static final double FRECUENCIA_OBJETIVO = FRECUENCIA;
    public static final double MINIMO = 60.0, MAXIMO = 120.0, TOLERANCIA =
0.3;
    public double frecuenciaMostrada(double valorDial); // dial INVERTIDO
    public double intensidadSenal(double valorDial);    // 0.0 lejos … 1.0 en
la frecuencia
    public boolean estaSintonizada(double valorDial);
    public String formatear(double frecuencia);         // "66.6 MHz"
    public String textoAyuda();                         // pista que se
muestra a los 60 s
}

// (4) logica/RelojPiedad.java
public class RelojPiedad {
    public RelojPiedad(javafx.util.Duration espera, Runnable alAgotarse);
    public void iniciar();        // empieza (o reinicia) la cuenta desde
cero
    public void parar();
    public boolean estaEnMarcha();
}

// (5) sonido/GestorSonido.java
public class GestorSonido {
    public GestorSonido();                              // carga audio/; si
falta algo, avisa y sigue sin ese sonido
    public void iniciarAmbiente();                      // MediaPlayer en bucle:
```

```
ambiente + estática
    public void actualizarSenal(double intensidad);  // 0..1: menos estática,
más voz lejana
    public void chasquido();                          // AudioClip al mover el
dial (limitado)
    public void reproducirVoz();                      // la voz fantasma que
dice la letra
    public void golpeSusto();                         // AudioClip del susto
    public void reiniciar();                          // vuelve a los
volúmenes iniciales
    public void detenerTodo();                        // al cerrar la app
}

// (6) efectos/Efectos.java
public final class Efectos {
    public static void respirar(Node nodo);               // pulso lento
infinito, sin destellos
    public static void temblar(Node nodo, double fuerza);  // sacudida corta,
fuerza 0..1
    public static void aparecer(Node nodo);               // fundido de
entrada suave
    public static void revelarLetra(Node letra, Duration visible, Runnable
alTerminar);
    public static void sustoControlado(Node susto);       // escala +
fundido, sin parpadeos
    public static void ocultar(Node... nodos);            // opacidad 0,
invisible, escala 1
    public static void detenerTodas();                    // para todo y
deja los nodos en posición neutra
}
```

Cómo los usa `SalaController` (parte 1)

1. **Al arrancar y al reiniciar:** `Efectos.detenerTodas()`, `Efectos.ocultar(lblAyuda, lblLetra, imgSusto)`, dial a un valor lejano, `sonido.reiniciar()`, `Efectos.respirar(lblTitulo)`.
2. **Al mover el dial:** muestra `formatear(frecuenciaMostrada(v))`, pone `pbSenal` a `intensidadSenal(v)`, llama a `sonido.actualizarSenal(...)` y `sonido.chasquido()`. Con el primer movimiento arrancan dos `RelojPiedad`: ayuda a los 60 s y letra a los 90 s.
3. **Al sintonizar:** para los relojes, `sonido.reproducirVoz()`, `Efectos.temblar(raiz, 0.5)`, `Efectos.sustoControlado(imgSusto)` + `sonido.golpeSusto()`, `Efectos.revelarLetra(lblLetra, 10 s, this::reiniciar)`.

4. **A los 60 s:** `lblAyuda` con `textoAyuda()` y `Efectos.aparecer(lblAyuda)`. **A los 90 s:** se apiada y revela la letra igual que en el paso 3.
5. **F12:** reinicio manual. **Ctrl+Shift+Q:** salir (solo para el equipo).

Prompt 2 · Vista FXML

Para el compañero 2. Necesita **Scene Builder** (Gluon, versión para JavaFX 21). Copiar entero en su IA:

```
CONTEXTO
Somos un grupo de 6 alumnos de 2.º de DAM (módulo Desarrollo de Interfaces).
Hacemos una aplicación JavaFX para el túnel del terror del instituto: la sala
2, "Radio fantasma". El visitante llega con una frecuencia que le da una
página HTML (FRECUENCIA MHz), gira un dial (Slider) hasta sintonizarla y
entonces suena una voz fantasma y aparece la letra "I". La interfaz rompe a
propósito un principio de usabilidad para dar miedo (el dial está invertido:
hacia la derecha baja la frecuencia y no tiene marcas), pero se tiene que
entender en 5 segundos qué hay que tocar. Prohibido cualquier destello rápido
o efecto estroboscópico.

Stack: Java 21, Maven, JavaFX 21 (controls, fxml, media), proyecto modular
con module-info.java. Se ejecuta con: mvn clean javafx:run. Paquete base:
es.medac.USUARIO.tunel. Recursos en
src/main/resources/es/medac/USUARIO/tunel/. Editor: VS Code. La app arranca
en pantalla completa; para salir mientras pruebo: Ctrl+Shift+Q.

Trabajamos 6 personas en paralelo y cada una solo toca SUS archivos. La rama
develop ya tiene un esqueleto que compila con todos los archivos y sus firmas
públicas. Yo trabajo en la rama feature/vista-fxml y al terminar abro un Pull
Request hacia develop. Commits convencionales (feat:, style:, fix:,
refactor:), mínimo 2. Si para probar necesito tocar otro archivo, lo hago
solo en local y lo descarto con git restore antes de commitear.

MI PARTE: la vista sala.fxml, hecha con Scene Builder y retocada a mano.
Archivos que me pertenecen (solo puedo tocar estos):
- src/main/resources/es/medac/USUARIO/tunel/sala.fxml
- src/main/resources/es/medac/USUARIO/tunel/img/radio*.png (opcional,
imágenes decorativas de la radio)

REQUISITOS OBLIGATORIOS DEL FXML (otros dependen de ellos, no se pueden
cambiar):
- Raíz: StackPane con fx:id="raiz", styleClass="raiz" y
fx:controller="es.medac.USUARIO.tunel.SalaController". Tamaño preferido
1280x720, pero tiene que verse bien a pantalla completa en cualquier
resolución: nada de posiciones absolutas, usa alineaciones y márgenes.
```

```
- Primer hijo de raiz: VBox fx:id="panelRadio" (styleClass "radio"),
centrado, que contiene en este orden:
  - Label fx:id="lblTitulo" (styleClass "titulo"): nombre de una emisora
fantasma inventada. NO debe revelar la frecuencia.
  - Label fx:id="lblInstruccion" (styleClass "instruccion"): una frase corta
y clara tipo "Gira el dial. Encuentra la frecuencia. Escúchala."
  - Label fx:id="lblFrecuencia" (styleClass "frecuencia"): texto inicial "-
-.- MHz".
  - Slider fx:id="sldDial" (styleClass "dial"): sin marcas ni etiquetas
(showTickMarks y showTickLabels a false). NO pongas min/max: los pone el
controlador.
  - ProgressBar fx:id="pbSenal" (styleClass "senal"), progress inicial 0,
ancho amplio.
  - Label fx:id="lblAyuda" (styleClass "ayuda"), visible="false",
wrapText="true".
- Encima de la radio (hijos siguientes de raiz, en este orden, para que se
pinten por encima):
  - Label fx:id="lblLetra" (styleClass "letra"), texto "I", visible="false",
mouseTransparent="true".
  - ImageView fx:id="imgSusto", visible="false", mouseTransparent="true",
preserveRatio="true", fitHeight unos 600, con <Image url="@img/susto.png"/>
(el archivo susto.png ya existe como provisional; lo cambia otro compañero).
- NADA de estilos en línea (ni style="..." ni fuentes ni colores en el FXML):
todo el aspecto lo pone otro compañero en estilos.css usando las styleClass.
Si añades nodos decorativos (rejilla de altavoz, marco, tornillos...), usa
solo las styleClass "altavoz" o "decoracion", sin fx:id.
- No añadas eventos (onAction, etc.) en el FXML: el controlador conecta todo
por código.

QUÉ TE PIDO
1. Explícame cómo abrir y editar sala.fxml en Scene Builder y cómo asignar
fx:id y styleClass.
2. Dame el sala.fxml completo cumpliendo todo lo anterior, con un diseño de
radio antigua que dé mal rollo (distribución, espaciados, alineaciones).
3. Explícame qué hace cada parte del FXML, porque estoy aprendiendo JavaFX.
4. Dime cómo comprobar que arranca con mvn clean javafx:run y qué errores
típicos salen si un fx:id no coincide con el controlador.
5. Propón 2-3 mensajes de commit convencionales para mi trabajo.
```

Prompt 3 · Estilo CSS y fuente

Para el compañero 3. Copiar entero en su IA:

CONTEXTO
Somos un grupo de 6 alumnos de 2.º de DAM (módulo Desarrollo de Interfaces). Hacemos una aplicación JavaFX para el túnel del terror del instituto: la sala 2, "Radio fantasma". El visitante llega con una frecuencia que le da una página HTML (FRECUENCIA MHz), gira un dial (Slider) hasta sintonizarla y entonces suena una voz fantasma y aparece la letra "I". La interfaz rompe a propósito un principio de usabilidad para dar miedo (el dial está invertido y no tiene marcas), pero se tiene que entender en 5 segundos qué hay que tocar. Prohibido cualquier destello rápido o efecto estroboscópico.

Stack: Java 21, Maven, JavaFX 21 (controls, fxml, media), proyecto modular con module-info.java. Se ejecuta con: mvn clean javafx:run. Paquete base: es.medac.USUARIO.tunel. Recursos en src/main/resources/es/medac/USUARIO/tunel/. Editor: VS Code. La app arranca en pantalla completa; para salir mientras pruebo: Ctrl+Shift+Q.

Trabajamos 6 personas en paralelo y cada una solo toca SUS archivos. La rama develop ya tiene un esqueleto que compila con todos los archivos y sus firmas públicas, que NO puedo cambiar (puedo añadir cosas privadas). Yo trabajo en la rama feature/estilo-css y al terminar abro un Pull Request hacia develop. Commits convencionales (feat:, style:, fix:, refactor:), mínimo 2. Si para probar necesito tocar otro archivo, lo hago solo en local y lo descarto con git restore antes de commitear.

MI PARTE: el tema oscuro en CSS y la carga de la fuente.
Archivos que me pertenecen (solo puedo tocar estos):
- src/main/resources/es/medac/USUARIO/tunel/estilos.css
- src/main/resources/es/medac/USUARIO/tunel/fuentes/ (los .ttf)
- src/main/java/es/medac/USUARIO/tunel/estilo/Fuentes.java

LO QUE YA HAY Y NO ES MÍO
- App.java llama a Fuentes.cargar() ANTES de crear la escena y luego añade estilos.css a la escena con getStylesheets().add(...). No tengo que tocar App.java.
- El FXML usa estas styleClass (son las únicas que tengo que estilar):
  raiz (StackPane raíz, fondo de toda la pantalla), radio (VBox carcasa de la radio), titulo (Label, nombre de la emisora), instruccion (Label con la instrucción; TIENE que leerse bien, es lo que hace que se entienda en 5 s), frecuencia (Label tipo display "87.3 MHz"), dial (Slider), senal (ProgressBar de la señal), ayuda (Label de ayuda que aparece a los 60 s), letra (Label con la letra "I" gigante encima de todo), altavoz y decoracion (nodos decorativos opcionales).
- El controlador añade la clase "sintonizada" al nodo raiz cuando aciertan, así que puedo estilar .raiz.sintonizada (por ejemplo cambiar el color del panel o del borde). Sin parpadeos.

```
FIRMA OBLIGATORIA DE Fuentes.java (paquete es.medac.USUARIO.tunel.estilo):
public final class Fuentes {
    public static void cargar(); // carga cada .ttf de fuentes/ con
Font.loadFont
}
- Cargar con
Font.loadFont(Fuentes.class.getResourceAsStream("/es/medac/USUARIO/tunel/fuen
tes/NOMBRE.ttf"), 20).
- Si devuelve null o el recurso no existe, avisar por consola y seguir (que
la app no se caiga).
- Imprimir por consola el getFamily() de cada fuente cargada: ese es el
nombre exacto que hay que poner en -fx-font-family en el CSS.

REQUISITOS
- Tema oscuro de radio antigua embrujada: fondo casi negro, madera/baquelita
vieja o metal oxidado, verde o ámbar de display antiguo, rojo sangre para la
letra. Puede usar gradientes, bordes, -fx-effect con dropshadow/innershadow
(estáticos).
- Una o dos fuentes de Google Fonts con licencia OFL (por ejemplo una para el
título y otra tipo display/LCD para la frecuencia). Apunta nombre, URL y
licencia para los créditos del README.
- Estilar el Slider por dentro (.dial .track, .dial .thumb) para que parezca
un dial de radio, y la ProgressBar (.senal .track, .senal .bar) como un
medidor de señal.
- La letra "I" muy grande (unos 250px), legible, con sombra.
- Nada que parpadee ni cambie de color rápido. JavaFX CSS no tiene
animaciones; las animaciones las hace otro compañero en Java.
- Contraste suficiente en .instruccion y .ayuda.

QUÉ TE PIDO
1. Dime qué fuente(s) bajar de Google Fonts, dónde poner el .ttf y cómo saber
su nombre de familia.
2. Dame Fuentes.java completo.
3. Dame estilos.css completo, comentado por secciones.
4. Explícame las diferencias entre CSS web y CSS de JavaFX (prefijo -fx-,
selectores de subestructura como .slider .thumb), porque estoy aprendiendo.
5. Propón 2-3 mensajes de commit convencionales y el texto de créditos de las
fuentes para el PR.
```

Prompt 4 · Lógica del dial y reloj de piedad

Para el compañero 4. Copiar entero en su IA:

```
CONTEXTO
Somos un grupo de 6 alumnos de 2.º de DAM (módulo Desarrollo de Interfaces).
Hacemos una aplicación JavaFX para el túnel del terror del instituto: la sala
2, "Radio fantasma". El visitante llega con una frecuencia que le da una
página HTML (FRECUENCIA MHz), gira un dial (Slider) hasta sintonizarla y
entonces suena una voz fantasma y aparece la letra "I". La interfaz rompe a
propósito un principio de usabilidad para dar miedo: el dial está INVERTIDO
(al moverlo a la derecha la frecuencia baja), pero la barra de señal ayuda a
entender en 5 segundos qué hacer. Siempre se tiene que poder ganar: a los 60
s se muestra una ayuda y a los 90 s la sala "se apiada" y da la letra.

Stack: Java 21, Maven, JavaFX 21 (controls, fxml, media), proyecto modular
con module-info.java. Se ejecuta con: mvn clean javafx:run. Paquete base:
es.medac.USUARIO.tunel. Editor: VS Code. La app arranca en pantalla completa;
para salir mientras pruebo: Ctrl+Shift+Q.

Trabajamos 6 personas en paralelo y cada una solo toca SUS archivos. La rama
develop ya tiene un esqueleto que compila con todos los archivos y sus firmas
públicas, que NO puedo cambiar (puedo añadir constantes y métodos privados).
Yo trabajo en la rama feature/logica-dial y al terminar abro un Pull Request
hacia develop. Commits convencionales (feat:, fix:, refactor:, test:), mínimo
2. No puedo tocar pom.xml. Si para probar necesito tocar otro archivo, lo
hago solo en local y lo descarto con git restore antes de commitear.

MI PARTE: la lógica de la sala, separada de la vista.
Archivos que me pertenecen (solo puedo tocar estos):
- src/main/java/es/medac/USUARIO/tunel/logica/Sintonizador.java
- src/main/java/es/medac/USUARIO/tunel/logica/RelojPiedad.java

FIRMAS OBLIGATORIAS (las usa SalaController, que es de otro compañero):

package es.medac.USUARIO.tunel.logica;
public class Sintonizador {   // Java puro, SIN imports de JavaFX
    public static final double FRECUENCIA_OBJETIVO = FRECUENCIA;
    public static final double MINIMO = 60.0, MAXIMO = 120.0, TOLERANCIA =
0.3;
    public double frecuenciaMostrada(double valorDial);
    public double intensidadSenal(double valorDial);
    public boolean estaSintonizada(double valorDial);
    public String formatear(double frecuencia);
    public String textoAyuda();
}

package es.medac.USUARIO.tunel.logica;
public class RelojPiedad {
    public RelojPiedad(javafx.util.Duration espera, Runnable alAgotarse);
```

```
    public void iniciar();
    public void parar();
    public boolean estaEnMarcha();
}

CÓMO LO USA EL CONTROLADOR
- El Slider va de MINIMO a MAXIMO. Cada vez que se mueve, el controlador
llama con su valor (valorDial) a frecuenciaMostrada, intensidadSenal y
estaSintonizada.
- Crea dos RelojPiedad: uno de 60 s que muestra textoAyuda() y otro de 90 s
que da la letra. Llama a iniciar() con el primer movimiento del dial y a
parar() cuando aciertan o al reiniciar la sala.

COMPORTAMIENTO QUE QUIERO
- frecuenciaMostrada: dial invertido, MINIMO + MAXIMO - valorDial.
- intensidadSenal: 1.0 si la distancia entre la frecuencia mostrada y la
objetivo es <= TOLERANCIA; fuera de eso baja de forma suave (curva, no
lineal) hasta 0.0 a unos 8 MHz de distancia. Resultado siempre entre 0 y 1.
- estaSintonizada: distancia <= TOLERANCIA.
- formatear: una cifra decimal con punto, por ejemplo "66.6 MHz" (Locale.US),
igual que la escribe la página HTML de 1.º.
- textoAyuda: una pista inquietante, calculada a partir de
FRECUENCIA_OBJETIVO (no escrita a mano), que diga entre qué dos decenas está
(por ejemplo "entre 60 y 70") y que el dial "miente" y hay que girarlo al
revés.
- Comprobar al crear el Sintonizador que FRECUENCIA_OBJETIVO está entre
MINIMO y MAXIMO (si no, IllegalStateException con un mensaje claro).
- RelojPiedad con PauseTransition: iniciar() para la cuenta anterior y
empieza de cero (playFromStart); al terminar ejecuta alAgotarse; parar() la
detiene sin ejecutar nada; estaEnMarcha() mira el estado de la animación. Se
usa siempre desde el hilo de JavaFX.

QUÉ TE PIDO
1. Dame Sintonizador.java y RelojPiedad.java completos, con Javadoc en
español.
2. Explícame la matemática de la intensidad y por qué se compara con
tolerancia y no con == en doubles.
3. Explícame qué es PauseTransition, el hilo de JavaFX y por qué RelojPiedad
no puede usar Thread.sleep.
4. Dame una forma de probar Sintonizador sin tocar pom.xml (por ejemplo,
jshell o un main temporal que no se commitea) con varios valores de ejemplo.
5. Propón 2-3 mensajes de commit convencionales.
```

Prompt 5 · Sonido

Para el compañero 5. Le tocará grabar la voz (con el móvil vale) y buscar sonidos libres. Copiar entero en su IA:

```
CONTEXTO
Somos un grupo de 6 alumnos de 2.º de DAM (módulo Desarrollo de Interfaces).
Hacemos una aplicación JavaFX para el túnel del terror del instituto: la sala
2, "Radio fantasma". El visitante llega con una frecuencia que le da una
página HTML (FRECUENCIA MHz), gira un dial (Slider) hasta sintonizarla y
entonces suena una voz fantasma y aparece la letra "I". Mientras busca, la
radio suena a estática y, cuanto más cerca está de la frecuencia, menos
estática y más susurros. Normas de seguridad: volumen alto pero no dañino,
nada de golpes saturados.

Stack: Java 21, Maven, JavaFX 21 (controls, fxml, media), proyecto modular
con module-info.java (ya incluye requires javafx.media). Se ejecuta con: mvn
clean javafx:run. Paquete base: es.medac.USUARIO.tunel. Recursos en
src/main/resources/es/medac/USUARIO/tunel/. Editor: VS Code. La app arranca
en pantalla completa; para salir mientras pruebo: Ctrl+Shift+Q.

Trabajamos 6 personas en paralelo y cada una solo toca SUS archivos. La rama
develop ya tiene un esqueleto que compila con todos los archivos y sus firmas
públicas, que NO puedo cambiar (puedo añadir cosas privadas). Yo trabajo en
la rama feature/sonido y al terminar abro un Pull Request hacia develop.
Commits convencionales (feat:, fix:, refactor:, chore:), mínimo 2. Si para
probar necesito tocar otro archivo, lo hago solo en local y lo descarto con
git restore antes de commitear.

MI PARTE: todo el audio de la sala.
Archivos que me pertenecen (solo puedo tocar estos):
- src/main/java/es/medac/USUARIO/tunel/sonido/GestorSonido.java
- src/main/resources/es/medac/USUARIO/tunel/audio/ (todos los archivos de
audio)

FIRMA OBLIGATORIA (la usa SalaController, que es de otro compañero):
package es.medac.USUARIO.tunel.sonido;
public class GestorSonido {
    public GestorSonido();                          // carga todo; si falta
un archivo, avisa por consola y sigue sin ese sonido
    public void iniciarAmbiente();                  // arranca los bucles
    public void actualizarSenal(double intensidad);  // 0.0 lejos ... 1.0
sintonizado
    public void chasquido();                        // clic al mover el dial
    public void reproducirVoz();                    // la voz fantasma que
dice la letra
```

```
    public void golpeSusto();                          // golpe del susto
    public void reiniciar();                           // volúmenes iniciales,
para la voz
    public void detenerTodo();                         // para y libera todo al
cerrar la app
}

AUDIOS QUE NECESITO (en audio/)
- ambiente.mp3: drone grave y oscuro, en bucle con MediaPlayer (cycleCount
INDEFINITE).
- estatica.mp3: estática de radio, en bucle con MediaPlayer; volumen alto
cuando intensidad=0 y casi nada cuando intensidad=1.
- susurros.mp3: susurros lejanos en bucle con MediaPlayer; volumen 0 cuando
intensidad=0, subiendo hasta alto en 1.
- voz.mp3: la voz que dice la letra (por ejemplo "Iiii... la letra es la
I..."). La grabo yo con el móvil y la proceso en Audacity (bajar tono,
eco/reverb) para que suene fantasmal.
- chasquido.wav: clic corto de dial, con AudioClip.
- susto.wav: golpe grave del susto, con AudioClip, normalizado sin saturar.
Solo formatos que soporta JavaFX: MP3 o WAV PCM (OGG NO funciona). Los
sonidos de internet tienen que ser de licencia libre (CC0 o similar, por
ejemplo Pixabay o Freesound filtrando por CC0); necesito anotar título,
autor, URL y licencia de cada uno para los créditos del README.

REQUISITOS DEL CÓDIGO
- Los MediaPlayer como ATRIBUTOS de la clase (si son variables locales, el
recolector de basura los elimina y el sonido se corta).
- Cargar con
getClass().getResource("/es/medac/USUARIO/tunel/audio/NOMBRE").toExternalForm
(), comprobando null. Usar setOnError para avisar por consola.
- chasquido() limitado: como mucho uno cada 80 ms aunque el Slider dispare
muchos eventos (System.nanoTime).
- actualizarSenal con transiciones de volumen suaves; volumen máximo de
cualquier pista 0.8.
- reproducirVoz baja la estática mientras habla la voz.
- Todo se llama desde el hilo de JavaFX.

QUÉ TE PIDO
1. Explícame la diferencia entre Media, MediaPlayer y AudioClip y cuándo usar
cada uno.
2. Guíame para grabar y procesar voz.mp3 en Audacity paso a paso, y dónde
buscar los demás sonidos con licencia libre.
3. Dame GestorSonido.java completo, con comentarios en español.
4. Dime cómo probarlo en local sin commitear cambios en otros archivos.
5. Propón 2-3 mensajes de commit convencionales y la plantilla de créditos
para el PR.
```

Prompt 6 · Animaciones y susto controlado

Para el compañero 6. Copiar entero en su IA:

```
CONTEXTO
Somos un grupo de 6 alumnos de 2.º de DAM (módulo Desarrollo de Interfaces). Hacemos una aplicación JavaFX para el túnel del terror del instituto: la sala 2, "Radio fantasma". El visitante llega con una frecuencia que le da una página HTML (FRECUENCIA MHz), gira un dial (Slider) hasta sintonizarla y entonces la pantalla tiembla, aparece una cara fantasma (susto controlado) y se revela la letra "I". Tras unos segundos la sala se reinicia sola para el siguiente grupo.

NORMA DE SEGURIDAD MUY IMPORTANTE: nada de destellos, parpadeos rápidos ni efectos estroboscópicos (fotosensibilidad). El susto es solo por aparición y escala. Ninguna opacidad o color puede alternar más rápido de una vez cada ~1,5 s, y nada de pantallazos blancos.

Stack: Java 21, Maven, JavaFX 21 (controls, fxml, media), proyecto modular con module-info.java. Se ejecuta con: mvn clean javafx:run. Paquete base: es.medac.USUARIO.tunel. Recursos en src/main/resources/es/medac/USUARIO/tunel/. Editor: VS Code. La app arranca en pantalla completa; para salir mientras pruebo: Ctrl+Shift+Q.

Trabajamos 6 personas en paralelo y cada una solo toca SUS archivos. La rama develop ya tiene un esqueleto que compila con todos los archivos y sus firmas públicas, que NO puedo cambiar (puedo añadir cosas privadas). Yo trabajo en la rama feature/efectos y al terminar abro un Pull Request hacia develop. Commits convencionales (feat:, fix:, refactor:), mínimo 2. Si para probar necesito tocar otro archivo, lo hago solo en local y lo descarto con git restore antes de commitear.

MI PARTE: todas las animaciones (Transition/Timeline) y la imagen del susto. Archivos que me pertenecen (solo puedo tocar estos):
- src/main/java/es/medac/USUARIO/tunel/efectos/Efectos.java
- src/main/resources/es/medac/USUARIO/tunel/img/susto.png (ahora hay uno provisional; lo sustituyo)

FIRMA OBLIGATORIA (la usa SalaController, que es de otro compañero):
package es.medac.USUARIO.tunel.efectos;
import javafx.scene.Node; import javafx.util.Duration;
public final class Efectos {
    public static void respirar(Node nodo);
    public static void temblar(Node nodo, double fuerza);
    public static void aparecer(Node nodo);
    public static void revelarLetra(Node letra, Duration visible, Runnable
```

```
alTerminar);
    public static void sustoControlado(Node susto);
    public static void ocultar(Node... nodos);
    public static void detenerTodas();
}
```

CÓMO LO USA EL CONTROLADOR
- Al arrancar y en cada reinicio: detenerTodas(), luego ocultar(lblAyuda, lblLetra, imgSusto), luego respirar(lblTitulo).
- Al acertar: temblar(raiz, 0.5), sustoControlado(imgSusto) y revelarLetra(lblLetra, Duration.seconds(10), this::reiniciar).
- A los 60 s: aparecer(lblAyuda).

COMPORTAMIENTO QUE QUIERO
- respirar: pulso lento e infinito de opacidad entre 1.0 y 0.65, ciclo de 2-3 s, autoReverse.
- temblar: sacudida corta (unos 0,4 s en total) con TranslateTransition; desplazamiento máximo fuerza*15 px; al terminar vuelve a translate 0.
- aparecer: setVisible(true) y FadeTransition de 0 a 1 en ~1,2 s.
- sustoControlado: setVisible(true); ParallelTransition de ScaleTransition (de 0.3 a 1.1) + FadeTransition (0 a 1) en ~0,35 s; se queda ~1,5 s y se desvanece en ~0,8 s; al final queda invisible.
- revelarLetra: setVisible(true); escala de 0.2 a 1 + fundido de entrada en ~1,5 s (SequentialTransition/ParallelTransition); espera "visible"; fundido de salida de 1 s; y al terminar ejecuta alTerminar.run().
- ocultar: para cada nodo, opacidad 0, invisible, escala 1 y translate 0.
- detenerTodas: guardar en una lista estática todas las animaciones lanzadas desde Efectos (y los nodos afectados); pararlas todas, vaciar la lista y dejar esos nodos en posición neutra (translate 0, escala 1, opacidad 1). Importante para que no se solapen animaciones entre un grupo de visitantes y el siguiente.
- Todo se llama desde el hilo de JavaFX.

IMAGEN susto.png
- Cara o silueta fantasma inquietante, sin gore, PNG con fondo transparente, unos 800 px de alto. Dibujada por nosotros o de licencia libre (CC0); anotar autor, URL y licencia para los créditos.

QUÉ TE PIDO
1. Explícame la diferencia entre Transition y Timeline, y entre ParallelTransition y SequentialTransition.
2. Dame Efectos.java completo, con comentarios en español.
3. Explícame cómo garantizar que no haya destellos y cómo justificarlo en la reflexión del README.

```
4. Dime cómo probar los efectos en local sin commitear cambios en otros
archivos.
5. Propón 2-3 mensajes de commit convencionales y el texto de créditos de la
imagen para el PR.
```